

Stack

- Traffic & Edge: Route53, AWS Global Accelerator, App load balancer in every region
- Compute & Orchestration: EKS (Fargate pods only), AWS Argo CD capability
- Database: Aurora Global
- Security: AWS WAF, Inspector
- Queue: SQS
- Observability: Amazon Prometheus

Flows

- Users come in through Route 53, and that routes everything to Global Accelerator. From there, traffic goes to one of the healthy region ALB, which then goes to the EKS running Fargate pods.
- App pushes tasks to SQS, and picks them up and process them asynchronously. Pods access AWS services like sqs, S3, Secrets Manager, and ECR through private VPC endpoints
- Deployments are fully GitOps-based. Argo CD inside of EKS watches code repo, pulls the latest desired state, grabs container images from ECR, and syncs everything into the EKS clusters automatically.
- The pods in EKS connect through RDS Proxy, which then connects to the global Aurora setup.
- Prometheus collects EKS pod application metrics which is used for monitoring, alerting, and analyzing performance.

A. Compute & Architecture:

Design a containerized infrastructure on AWS EKS.

Your solution must demonstrate how the application remains available and responsive during extreme load events and infrastructure failures.

High Availability

App is available and responsive during infrastructure failures

- I chose an active-active multi-region setup with AWS EKS. This avoids relying on a single region and prevents any impact during regional outages as each EKS cluster is capable of serving requests independently.
- AWS Global Accelerator will be forwarding the traffic to the application. AWS Global Accelerator performs continuous health checks and automatically forwards traffic to the region

with the healthy EKS cluster for a fast regional failover. This keeps the application always responsive for users.

- Tradeoff: If global caching is required, CloudFront can be replaced with Global Accelerator. However, CloudFront is slower than global accelerator and has more complexity related to routing during failover to a different region.

B. Scalability Strategy

The system must automatically handle the 10x traffic spike scenario without manual intervention.

Define your strategy for scaling both the application and the underlying infrastructure.

Scalability

Automatic scaling of apps and infra during extreme load events such as 10x traffic spikes

- I decided on only running AWS Fargate pods with a Horizontal Pod Autoscaler (HPA). Application will automatically scale and efficiently handle any burst load. The HPA will increase or reduce the number of Fargate pods depending on resource usage and demand. This helps the platform handle traffic spikes efficiently and keeps operations simple. There is no bottleneck on any EC2 worker nodes being created.
- I decided to use Amazon SQS between services. This will decouple services and prevent any sudden traffic spikes from overwhelming the system. SQS will queue incoming tasks and allow workers to process them at a controlled rate as demand increases. This helps the platform scale more smoothly.
- Apps will use Amazon RDS Proxy with Amazon Aurora to manage database connections more efficiently as traffic scales. This helps reduce connection overload on Aurora during traffic spikes, and allows the database layer to scale more reliably.
- Amazon Aurora scales storage automatically as application data grows over time. This removes the need for manual storage management and helps the database handle increasing data volumes. Tradeoff: DynamoDB Global Tables can be used instead of Aurora Global Database to support multi-region writes, but does not guarantee atomicity for transactions.
- Tradeoff: KEDA, a more advanced kubernetes autoscaler, which supports more flexible metrics such as SQS queue length can also be used alongside default HPA but requires more AWS permissions.
- Tradeoff: ec2 nodes requires careful capacity and deployment planning although a separate worker ec2 nodegroup can be cost saving to handle regular load traffic.

C. Security & Networking

Design a network and security architecture that protects customer data and infrastructure integrity.

Apply "Least Privilege" and "Defense in Depth" principles to the entire stack.

Security in network architecture

Least privilege and defense in depth

- I chose to use AWS WAF on top of the AWS Application Load Balancer to protect the application from web attacks. This helps filter and block malicious traffic like SQL injection and bot requests before they even reach the application.
- pods will use AWS IAM Roles for Service Accounts so each pod running in Amazon Elastic Kubernetes Service gets its own limited permission to access aws services without any hard coded password. This improves security by ensuring each workload only has access to the specific AWS resources it actually needs.
- Amazon Inspector will scan ECR images for security vulnerabilities.
- All AWS services are accessed through private VPC endpoints, reducing internet data transfer costs and improving security.
- In the current vpc design, there is no public endpoint access to EKS, no ec2 or SSH access, or no application workloads in public subnets.

D. Reliability & Observability

Define how you will measure the health of the service

And how the system will recover from failures (e.g., AZ outage, bad deployment) with minimal impact to the customer.

Measure app health

- Prometheus which will collect SLIs such as latency, error rates, and availability from application pods. This allows tracking reliability in real time and quickly alert if the service is deviating from expected performance.

Recovery from failures such as incorrect deployment

- I will use Argo Rollouts and prometheus metrics for progressive delivery of new app versions. Argo shifts traffic to new deployments with slow canary releases, so if there is any issues then it is caught early without impacting all users at once and automatically rolled back.

E. Operations

Day 2 Operations: Propose strategies to minimize operational toil.

Task Assignment: Outline how the implementation tasks for this project should be assigned across a team of 3 engineers (1 Senior, 2 Juniors).

Minimize day 2 operations toil

- Every app build and infra change will be a through CI/CD pipeline. And apps will be managed with GitOps using AWS Argo CD capability. Argo CD automatically syncs cluster state from Git, so I don't have to manually deploy or manage changes in the environment.
- Earlier I explained about architectural decision for no need to manually manage any ec2 or database storage. No capacity management is required. Since there are no EC2 instances, there is no need for node patching.
- rds proxy and aws global accelerator automatically take care of any failover

Split tasks to 2 juniors and 1 senior engineers

- I decided to split responsibilities so one junior handles all Kubernetes tasks and another focuses on writing Terraform code. This helps reduce complexity and keeps each person focused on a single domain.
- The senior is then free and will review the juniors' work through pull requests, which keeps quality high and keeps developing the juniors. This setup also frees the senior to focus more on optimization and long-term architecture improvements instead of day-to-day work.